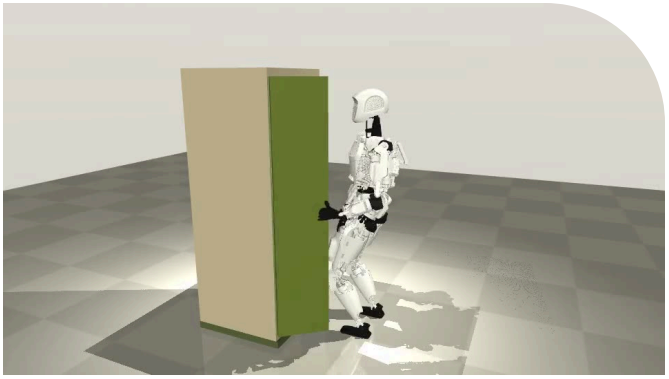
Capture a cabinet's motion. Get its URDF. Then a humanoid opens it.

ObjectModel turns a capture of an articulated object into a measured, MuJoCo-validated digital twin. **Twin-to-Task** hands that twin to a humanoid, either the public Apollo model or a generic upper body. The humanoid opens the object in a simulation of its *ground truth*, so the simulator scores whether the twin is good enough to act on.



Independent student project · not affiliated with Apptronik · Apollo model: Google DeepMind MuJoCo Menagerie (Apache-2.0)

Twin accuracy vs humanoid effort

Twin used to plan	Stiff arm	Extra force	Apollo	Extra path
ObjectModel twin	**100%**	0 N	**100%**	0 mm
+2° axis error	0%	19 N	**100%**	−1 mm
+5°	0%	50 N	**100%**	4 mm
+10°	0%	79 N	80%	22 mm

Doors, drawers and lids on 5 objects, each judged against the same task planned with the true joint (7 feasible on the stiff arm, 10 on Apollo). A stiff arm shows a bad twin as force; Apollo's compliant arm shows it as path error. Twins average 0.18° of axis error at 1 mm tracking noise; the stiff arm starts failing between 0.5° and 1°. Synthetic captures.

Under the hood

- RANSAC rigid segmentation; joint type chosen by BIC; screw-axis fits with bootstrap uncertainty
- Kinematic trees for chains and branches; mass and inertia; URDF and MJCF exports that agree to 0.00 mm
- MuJoCo checks: replay vs capture, collision sweep, stability
- Humanoid: affordances, a stance search over the handle arc, DLS IK on the reference pose (torso and 7-DoF arm on Apollo)
- C++/Eigen core that matches NumPy to 1e-16: revolute fit about 30× faster, 0.16 µs per DLS solve
- CLI, REST API, MCP server for AI agents, web playground, CI, 142 tests

Where it meets the roles in Apptronik's public postings

Developer experience	CLI, MCP server for AI coding agents, hosted API, CI, a dev server that mirrors production
Real-time controls	Frames and quaternions, DLS IK with nullspace posture, servo tracking under load, a C++ core with a sub-µs DLS solve
Motion planning	Upper-body IK on the Apollo model, stance planning, constraint-force metrics
Simulation and sim-to-real	Simulator-in-the-loop validation, parameter estimation from motion, noise and camera-sway sweeps, a clearance check that caught an idle-arm collision
Autonomy and perception	Perceive → model → plan → act → verify loop; LK tracking, depth lifting, ArUco scale. ROS 2 node is the next step.
Fleet and full-stack	Validated Python serverless API, a static web app with live 3D, deploy config

Run it: `objectmodel task kitchen_cabinet` · `objectmodel task-bench` · `objectmodel task fridge --robot apollo --video fridge.mp4` · Code: github.com/ayanbhardwaj1/objectmodel · Limits: contact-constraint grasp, pelvis held at stand, synthetic captures.